Elliptic Curve Digital Signature Algorithm (ECDSA)

Vediamo ora nel dettaglio come funziona l'algoritmo di firma digitale per chiavi basate su Elliptic Curve

Sia $(\mathbb{F}_p, \cdot, +)$ il campo dei numeri $\{0, 1, \dots, p-1\}$

Sia $C = \{(x,y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\infty\}$ una curva ellittica

Sia G un punto sulla curva t.c. l'ordine n del gruppo $\langle G \rangle$ sia primo

Sia (sk, pk) una coppia di chiavi t.c. $sk \in \{1, \dots, p-1\}$ e $pk = sk \cdot G$

Sia Hash una funzione crittografica

Algoritmo di firma digitale

Sign (sk, msg):

$h = Hash(msg)$ — si assume che $h < n$

Sceglie $k < n$ u.a.r. — generatore sicuro

$R = k \cdot G$ indicato con le coordinate (r, y_r)

$s = k^{-1}(h + r \cdot sk) \bmod n$

Return $\sigma = (r, s)$

Oss

È cruciale che k rimanga nascosto in quanto se scoperto permette di risalire alla sk

Algoritmo di Verifica

Verify $(msg, pk, \sigma = (r,s))$

$h = Hash(msg)$

$u_1 = s^{-1} h \bmod n$

$u_2 = s^{-1} r \bmod n$

$\tilde{R} = u_1 G + u_2 pk$

$s^{-1} h G + s^{-1} r pk = s^{-1} G (h + r \cdot sk) = \dfrac{kG(h + r \cdot sk)}{(h + r \cdot sk)} = kG = R$

Return True se la 1ª coordinata di $\tilde{R}$ è uguale a quella di R

Segregated Witness

Uno degli aggiornamenti più importanti di bitcoin è stato il BIP 144 rinominato Segregated Witness.
Prima di parlare del Bip in sé bisogna definire la differenza fra SoftFork e HardFork

SoftFork e HardFork

Per come è progettata la rete di BitCoin, questa è restia ad ogni aggiornamento visto che deve essere adottato da tutti i miner della rete.

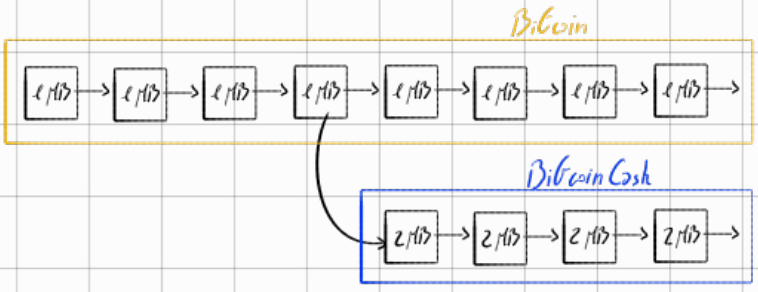
In ogni caso ogni aggiornamento è buona prassi che sia retrocompatibile in modo che non avvenga un fork della catena

Def

Con Hard Fork si intende una biforcazione della catena con le rispettive modifiche al protocollo di consenso

Esempio

Un HardFork è avvenuto quando una parte della comunità di miner decise di raddoppiare la dimensione dei blocchi andando a creare la catena BitCoin Cash



Def

Con SoftFork invece si intendono modifiche più restrittive al protocollo di consenso, che pur non essendo retrocompatibili non causano una biforcazione della catena

Parliamo ora delle modifiche apportate da SegWit alla catena

Motivazioni e modifiche

La motivazione principale che ha voluto risolvere SegWit è quella della Malleabilità delle Transazioni

Questa consiste in una leggera modifica della signature in modo che questa sia ancora valida ma l'Hash della Tx no

Per questo è stato deciso di modificare la struttura delle Tx in modo tale che

- gli ScriptSig degli input vengano spostati in una sezione differente della Tx chiamata Witness
- l'Hash della Tx venga calcolato escludendo i Witness → Così si è risolto il prob. della malleabilità

Tre SegWit

Versione	# input	input	# output	output	lockTime

Post SegWit

Identifica la Tx come SegWit

Versione	Marker	Flag	# output	output	lockTime	Witness

Definisce la presenza di Witness

Un'ulteriore modifica apportata al protocollo da SegWit è l'unità di misura del blocco.

Questo perché ci si era resi conto che la maggior parte dello spazio dei blocchi veniva occupato dalle signature,

per questo l'unità di misura del blocco è cambiata da 1 MB a 4'000'000 WU

→ Weigh Units

Successivamente hanno trattato diversamente i dati:

- Dati base 1 byte -> 4 WU
- Signature 1 byte -> 1 WU